MSA-DATB09-03 · NEURAL NETWORKS & DEEP LEARNING FOUNDATIONS

Session 3 / 8

Your First Neural Network and Backpropagation

Stack neurons into layers, solve XOR, derive backpropagation by hand, then let PyTorch do it.

Guillaume Saint-Cirgue Albert School · MSc 1 October 2026

Guillaume Saint-Cirgue your instructor

Data Scientist and ML Engineer since 2016

2013 to 2016	Engineering degree, **ENSMM** (Besançon)
2016 to 2017	Data Scientist, **Rolls-Royce**
2017 to 2019	Lead ML Engineer, **GKN Aerospace**
2019	Launched **Machine Learnia** on YouTube: 200,000+ subscribers today
2020	Founded my company and my startup: **30+ AI projects** delivered since
2023	Head of Data, **Airbus**
Today	**ML-PRO** training program: 800 students

To go further

MY YOUTUBE CHANNEL

Machine Learnia

200,000+ subscribers. Machine learning and deep learning videos, including a neural network coded from scratch in NumPy.

youtube.com/@MachineLearnia

MY EBOOK

Apprendre le Machine Learning en une semaine

In English: Learn Machine Learning in One Week.

machinelearnia.com/apprendre-le-machine-learning-en-une-semaine

Both resources are in French. Click the cards in your copy of the course to open them.

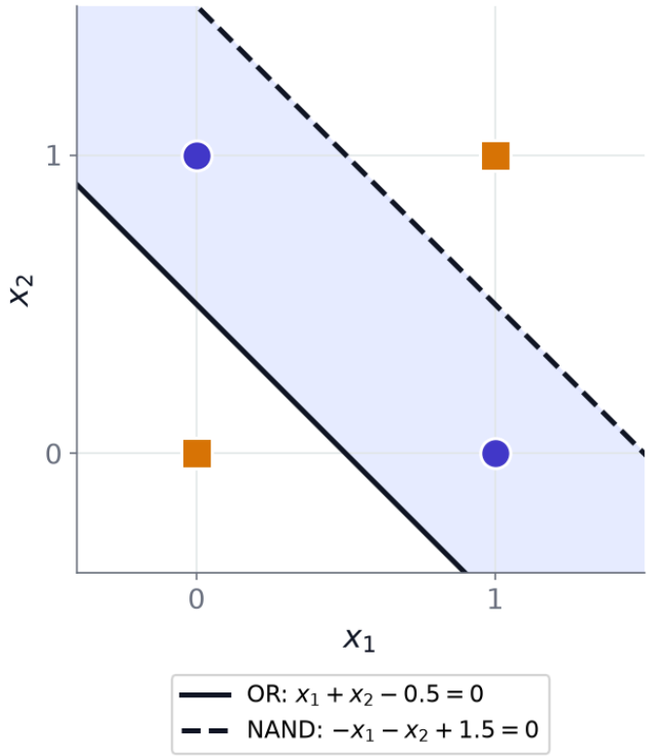
One neuron draws one line. Two neurons draw two lines.

Last two sessions: one neuron (perceptron or sigmoid) separates the plane with **one line**, so XOR is impossible.

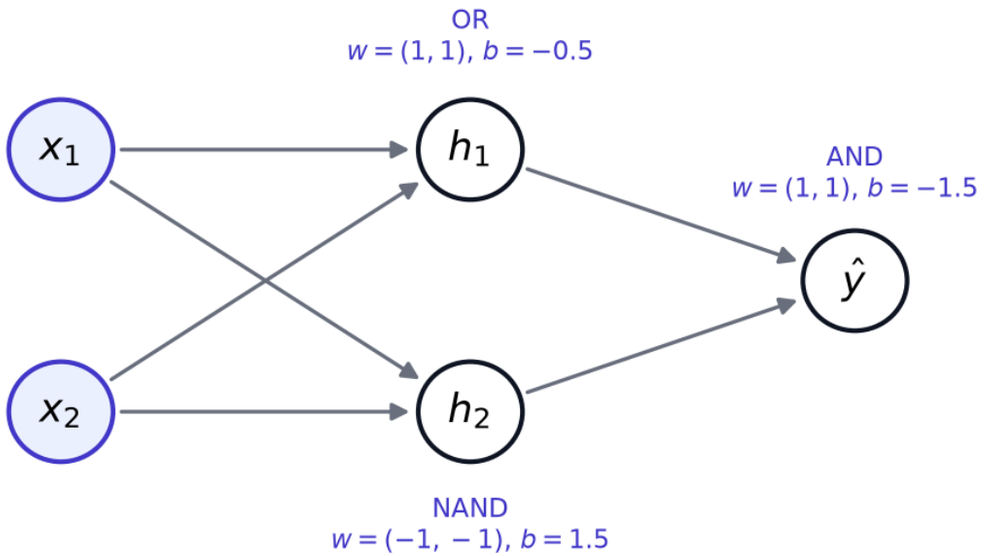
Idea: use **two** neurons, each drawing its own line, then a **third** neuron that combines their answers.

- OR line: $(0, 0)$ below, the 3 others above.
- NAND line: $(1, 1)$ above, the 3 others below.
- XOR = 1 exactly **between** the two lines: OR **and** NAND.

$$\mathrm{XOR}(x_1, x_2) = \mathrm{AND}\big(\mathrm{OR}(x_1, x_2),\ \mathrm{NAND}(x_1, x_2)\big)$$



Exercise 1 · Compute the XOR network by hand



Step activation, the weights of session 1.

Exercise 10 min pen and paper

1. For the 4 inputs $(0,0), (0,1), (1,0), (1,1)$, compute h_1, h_2 (step activation), then $\hat{y}$. Does the network compute XOR?
2. Now replace every step by a sigmoid and multiply all weights and biases by 10. Compute $h_1, h_2, \hat{y}$ for $x = (0,1)$. Use $\sigma(5) \approx 0.993$ and $\sigma(4.87) \approx 0.992$.

Your notes / answers (saved in this browser)

What the hidden layer does: a new representation

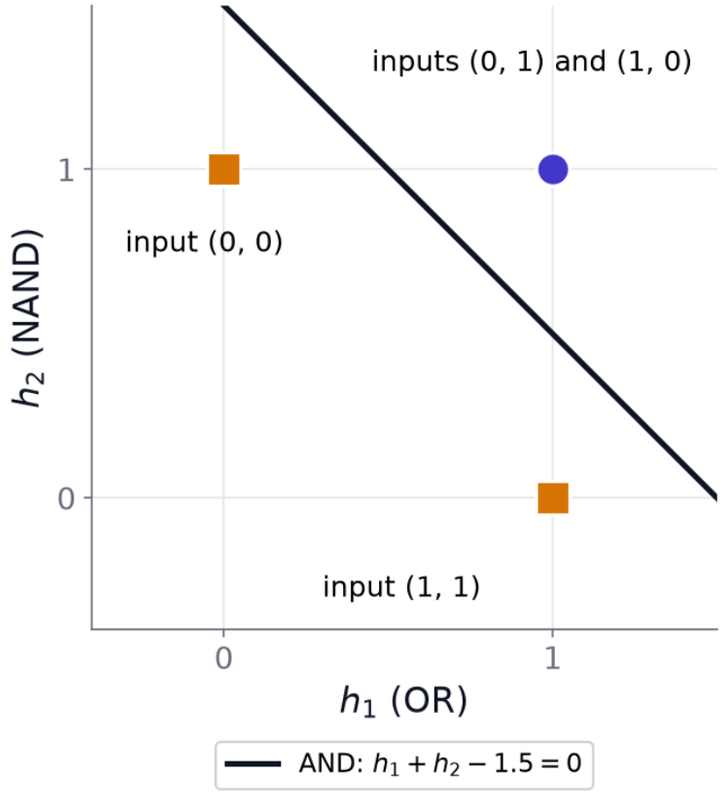
The two hidden neurons transform each input x into a new point $h = (h_1, h_2)$:

input x	hidden h	XOR
(0, 0)	(0, 1)	0
(0, 1), (1, 0)	(1, 1)	1
(1, 1)	(1, 0)	0

In the hidden space the two classes **are** separable by one line: the output neuron does the rest.

Key idea

A hidden layer learns new features in which the problem becomes easy for the last neuron. Training will find such weights by itself.

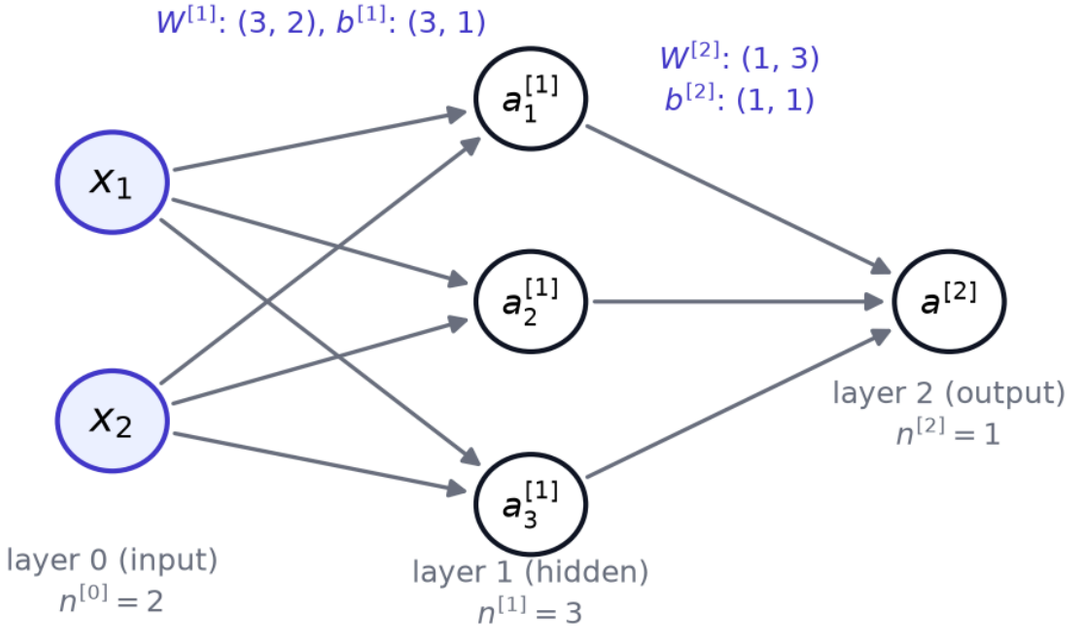


Vocabulary and shapes

- **Layer** l has $n^{[l]}$ neurons. Layer 0 is the input: $n^{[0]} = n$ features.
- Hidden layers are between the input and the **output layer**. Here: a 2-3-1 network.
- Neuron k of layer l has one weight per neuron of layer $l-1$. Stack them as rows of a matrix:

$$W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}, \qquad b^{[l]} \in \mathbb{R}^{n^{[l]}}$$

Number of parameters of layer l: $n^{[l]} \times n^{[l-1]} + n^{[l]} = n^{[l]}\,(n^{[l-1]} + 1)$.



Exercise 2 · Shapes and parameter counts

Exercise 7 min pen and paper

1. The 2-3-1 network of the figure: give the shapes of $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}$ and the total number of parameters.
2. A network for 28×28 grey images of digits: 784 inputs, a hidden layer of 128 neurons, 10 outputs (784-128-10). Same questions.
3. Which layer holds most of the parameters of the 784-128-10 network, and why?

Your notes / answers (saved in this browser)

Forward propagation

One example x (a column)

$$
\begin{aligned}
z^{[1]} &= W^{[1]} x + b^{[1]} && (n^{[1]}, 1) \\
a^{[1]} &= \sigma(z^{[1]}) && (n^{[1]}, 1) \\
z^{[2]} &= W^{[2]} a^{[1]} + b^{[2]} && (n^{[2]}, 1) \\
a^{[2]} &= \sigma(z^{[2]}) && (n^{[2]}, 1)
\end{aligned}
$$

Each layer does what one neuron did in session 2, for $n^{[l]}$ neurons at once. Its input is the output of the previous layer.

m examples (as columns of X)

$$
\begin{aligned}
A^{[0]} &= X && (n^{[0]}, m) \\
Z^{[1]} &= W^{[1]} A^{[0]} + b^{[1]} && (n^{[1]}, m) \\
A^{[1]} &= \sigma(Z^{[1]}) && (n^{[1]}, m) \\
Z^{[2]} &= W^{[2]} A^{[1]} + b^{[2]} && (n^{[2]}, m) \\
A^{[2]} &= \sigma(Z^{[2]}) && (n^{[2]}, m)
\end{aligned}
$$

$b^{[l]}$ is added to every column (broadcasting).

The same forward pass in code: examples are rows

NumPy and PyTorch put one example per **row**: X has shape $(m, n^{[0]})$. Transpose everything: $(WX)^\top = X^\top W^\top$, so in code

$$Z^{[l]} = A^{[l-1]}\, W^{[l]\top} + b^{[l]} \qquad \text{shape } (m, n^{[l]})$$

```
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])             # (m, n0) = (4, 2)
W1, b1 = np.array([[1, 1], [-1, -1]]), np.array([-0.5, 1.5])   # W1: (n1, n0) = (2, 2)
W2, b2 = np.array([[1, 1]]), np.array([-1.5])               # W2: (n2, n1) = (1, 2)
step = lambda z: (z ⩾ 0).astype(int)
A1 = step(X @ W1.T + b1)       # (4, 2): [[0 1] [1 1] [1 1] [1 0]]
A2 = step(A1 @ W2.T + b2)      # (4, 1): [[0] [1] [1] [0]], XOR
```

This is exactly what `nn.Linear(n_in, n_out)` computes: `X @ W.T + b` with `weight` of shape `(n_out, n_in)`.

Why a non-linear activation?

Remove the sigmoid: two linear layers in a row.

$$z^{[2]} = W^{[2]}\left(W^{[1]}x + b^{[1]}\right) + b^{[2]} = \underbrace{W^{[2]}W^{[1]}}_{W'}\,x + \underbrace{W^{[2]}b^{[1]} + b^{[2]}}_{b'}$$

- A stack of linear layers is **one** linear layer $W'x + b'$: it still draws a single line, however many layers we stack.
- The non-linear activation between the layers (sigmoid today; ReLU in session 5) is what lets the network bend its boundary.

Pitfall

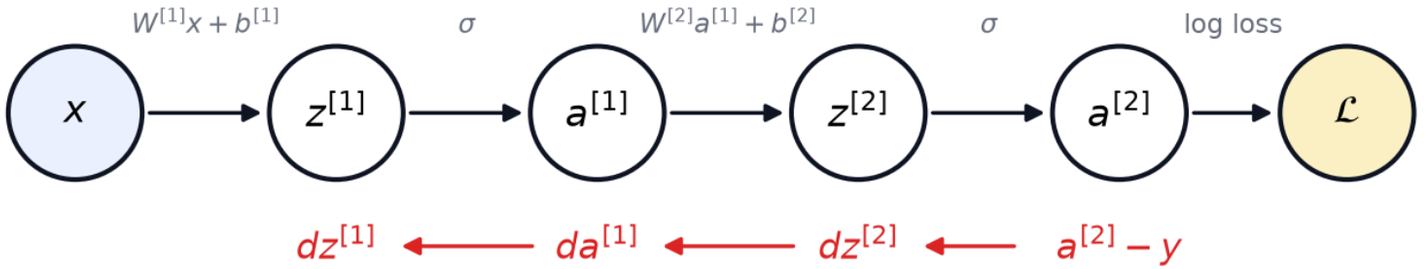
Forgetting the activation between two `nn.Linear` layers is a classic bug: the code runs, but the model is linear.

Backpropagation: the problem

To train with gradient descent we need $\partial \mathcal{L}/\partial W^{[1]}$ and $\partial \mathcal{L}/\partial W^{[2]}$. The output layer is easy: it is logistic regression on the input $a^{[1]}$. But a weight of layer 1 acts on the loss only **through** everything after it:

$$W^{[1]} \to z^{[1]} \to a^{[1]} \to z^{[2]} \to a^{[2]} \to \mathcal{L}$$

Chain rule along this path, starting from the end and reusing each result for the next step: that is **backpropagation** (Rumelhart, Hinton and Williams, 1986).



The computation graph of one example. Notation: $dz^{[l]} = \partial \mathcal{L}/\partial z^{[l]}$, $dW^{[l]} = \partial \mathcal{L}/\partial W^{[l]}$.

Backpropagation step 1: the output layer

One example, sigmoid output, log loss. Session 2 gave, with the input of the layer now being $a^{[1]}$ instead of x:

$$dz^{[2]} = \frac{\partial \mathcal{L}}{\partial z^{[2]}} = \frac{\partial \mathcal{L}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} = \frac{a^{[2]} - y}{a^{[2]}(1 - a^{[2]})} \cdot a^{[2]}(1 - a^{[2]}) = a^{[2]} - y$$

Then, since $z^{[2]} = \sum_k W_k^{[2]}\, a_k^{[1]} + b^{[2]}$:

$$\frac{\partial \mathcal{L}}{\partial W_k^{[2]}} = dz^{[2]}\, a_k^{[1]} \quad \Longrightarrow \quad dW^{[2]} = dz^{[2]}\, a^{[1]\top}, \qquad db^{[2]} = dz^{[2]}$$

Error of the layer times its input, as in session 2. Shapes: $dz^{[2]}$ is $(1, 1)$, $a^{[1]\top}$ is $(1, n^{[1]})$, so $dW^{[2]}$ has the shape of $W^{[2]}$.

Backpropagation step 2: back to the hidden layer

Weight $W_{kj}^{[1]}$ (from input j to hidden neuron k) acts through $z_k^{[1]} \to a_k^{[1]} \to z^{[2]}$:

$$\frac{\partial \mathcal{L}}{\partial W_{kj}^{[1]}} = \underbrace{\frac{\partial \mathcal{L}}{\partial z^{[2]}}}_{dz^{[2]}} \cdot \underbrace{\frac{\partial z^{[2]}}{\partial a_k^{[1]}}}_{W_k^{[2]}} \cdot \underbrace{\frac{\partial a_k^{[1]}}{\partial z_k^{[1]}}}_{a_k^{[1]}(1-a_k^{[1]})} \cdot \underbrace{\frac{\partial z_k^{[1]}}{\partial W_{kj}^{[1]}}}_{x_j}$$

The first three factors form $dz_k^{[1]}$. For all hidden neurons at once ($\odot$ = element-wise product):

$$dz^{[1]} = \underbrace{W^{[2]\top} dz^{[2]}}_{da^{[1]}:\,(n^{[1]},1)} \odot\ a^{[1]} \odot (1 - a^{[1]}), \qquad dW^{[1]} = dz^{[1]}\, x^\top, \qquad db^{[1]} = dz^{[1]}$$

$W^{[2]\top}$ sends the error of the output back to each hidden neuron, weighted by the connection it used on the way forward.

The full backward pass, vectorised over m examples

$$
\begin{aligned}
dZ^{[2]} &= A^{[2]} - Y \\
dW^{[2]} &= \frac{1}{m}\, dZ^{[2]} A^{[1]\top} \qquad db^{[2]} = \frac{1}{m} \sum_{\text{columns}} dZ^{[2]} \\
dZ^{[1]} &= W^{[2]\top} dZ^{[2]} \odot A^{[1]} \odot \left(1 - A^{[1]}\right) \\
dW^{[1]} &= \frac{1}{m}\, dZ^{[1]} X^{\top} \qquad db^{[1]} = \frac{1}{m} \sum_{\text{columns}} dZ^{[1]}
\end{aligned}
$$

The pattern, for any layer

$dZ^{[l]} = W^{[l+1]\top} dZ^{[l+1]} \odot \sigma'(Z^{[l]})$ and $dW^{[l]} = \frac{1}{m}\, dZ^{[l]} A^{[l-1]\top}$. The error flows backwards, one layer at a time.

quantity	shape
$dZ^{[2]}$, $A^{[2]}$	$(n^{[2]}, m)$
$dW^{[2]}$	$(n^{[2]}, n^{[1]})$
$dZ^{[1]}$, $A^{[1]}$	$(n^{[1]}, m)$
$dW^{[1]}$	$(n^{[1]}, n^{[0]})$
X	$(n^{[0]}, m)$

Each gradient has the shape of its parameter: a good check when you code it.

Exercise 3 · Forward and backward by hand

A 2-2-1 network with sigmoids, one example $x = (1, 2)$, label $y = 1$:

$$W^{[1]} = \begin{pmatrix} 1 & -1 \\ 0.5 & 0.5 \end{pmatrix}, \quad b^{[1]} = \begin{pmatrix} 1 \\ -1.5 \end{pmatrix}, \qquad W^{[2]} = \begin{pmatrix} 1 & -1 \end{pmatrix}, \quad b^{[2]} = 0$$

Exercise 15 min pen and paper

1. Forward: $z^{[1]}, a^{[1]}, z^{[2]}, a^{[2]}$ and the loss $\mathcal{L}$.
2. Backward: $dz^{[2]}$, $dW^{[2]}$, $db^{[2]}$, then $da^{[1]}$, $dz^{[1]}$, $dW^{[1]}$, $db^{[1]}$.
3. One gradient step with $\eta = 1$: new $W_1^{[2]}$ and new $W_{11}^{[1]}$.
4. Bonus: update all parameters and compute the new $a^{[2]}$ ($\sigma(0.75) \approx 0.679$, $\sigma(1.108) \approx 0.752$).

Your notes / answers (saved in this browser)

Break: 10 minutes

Autograd does exactly this

```
import torch
x, y = torch.tensor([1.0, 2.0]), torch.tensor([1.0])
W1 = torch.tensor([[1.0, -1.0], [0.5, 0.5]], requires_grad=True)
b1 = torch.tensor([1.0, -1.5], requires_grad=True)
W2 = torch.tensor([[1.0, -1.0]], requires_grad=True)
b2 = torch.tensor([0.0], requires_grad=True)
a1 = torch.sigmoid(W1 @ x + b1)                # forward: PyTorch records the graph
a2 = torch.sigmoid(W2 @ a1 + b2)
loss = -(y * torch.log(a2) + (1 - y) * torch.log(1 - a2)).sum()
loss.backward()                                # backward: the chain rule, layer by layer
print(W2.grad)        # tensor([[-0.2500, -0.2500]])
print(W1.grad)        # tensor([[-0.1250, -0.2500], [ 0.1250,  0.2500]])
```

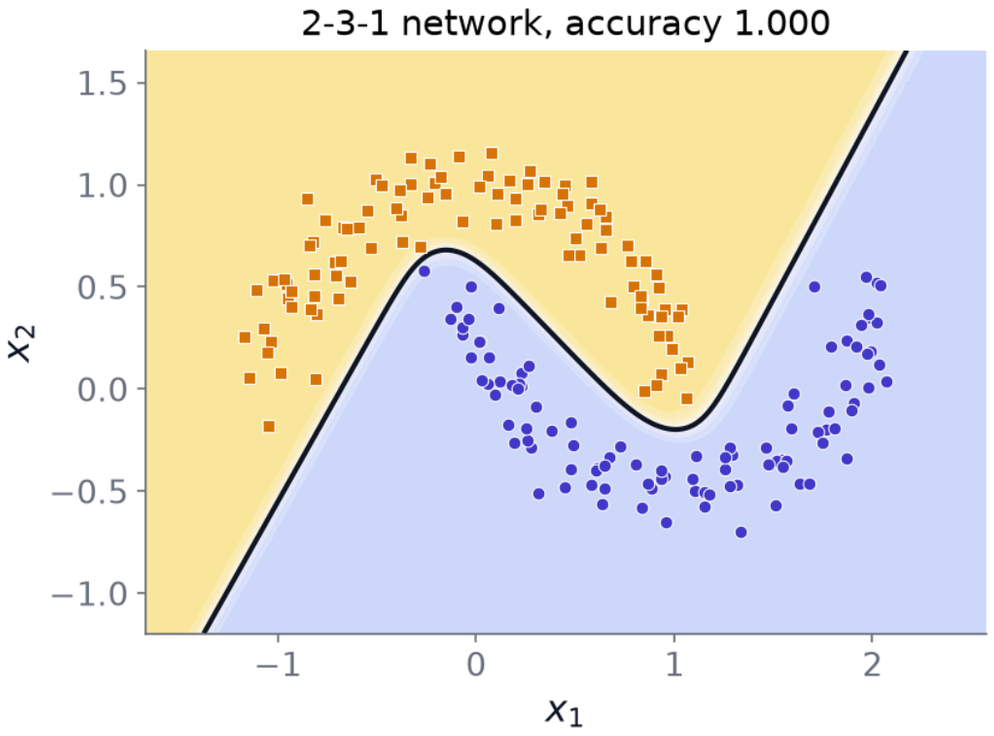
- During the forward pass, every operation on a tensor with `requires_grad=True` is recorded in a graph.
- `backward()` walks this graph from the loss back to the parameters (reverse mode) and applies the local derivatives we wrote by hand. Same numbers as exercise 3.

The network in PyTorch

```
model = nn.Sequential(
    nn.Linear(2, 3),     # layer 1: weight (3, 2), bias (3,)
    nn.Sigmoid(),        # the non-linearity
    nn.Linear(3, 1),     # layer 2: weight (1, 3), bias (1,): the logit
)
loss_fn = nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=5.0)
for epoch in range(2000):                    # the loop of session 2
    optimizer.zero_grad()
    loss = loss_fn(model(X_t).squeeze(1), y_t)
    loss.backward()                          # backpropagation, both layers
    optimizer.step()
print(sum(p.numel() for p in model.parameters()))   # 13
```

- `nn.Sequential` chains layers: the output of one is the input of the next.
- No sigmoid at the end: `BCEWithLogitsLoss` applies it (session 2).
- The training loop is unchanged (`X_t`, `y_t`: the moons as float32 tensors): only the model is bigger.
- $\eta = 5$ is large, but fine here: tiny dataset, full batch, and the sigmoid derivatives (at most 0.25) shrink the gradients.

The boundary now bends



- Same moons as session 2. Logistic regression: 0.885. The 2-3-1 network: **1.000** on the training set.
- Each hidden neuron draws a line; the output neuron combines the three into a curved boundary.
- More hidden neurons: more flexible boundaries (TP Part D).

Training accuracy only: whether it generalises needs a test set (Going further, and session 4).

TP · Your first neural network

TP 78 min tp/TP03_first_neural_network.ipynb

1. Part A (15 min): the XOR network with the weights of exercise 1, in NumPy.
2. Part B (25 min): forward and backward of exercise 3 in NumPy, checked with finite differences and with autograd.
3. Part C (25 min): `nn.Sequential` trained on XOR, moons and circles.
4. Part D (13 min): the effect of the number of hidden neurons.

Deliverable: every cell prints "Exercise validated!".

Your notes / answers (saved in this browser)

Watch it on YouTube Machine Learnia, Deep Learning series

Today's network and backpropagation, plus the full NumPy version we do not code in class.

Video 7

Réseau de neurones à 2 couches

A neural network with 2 layers

Video 8

La backpropagation en détail

Backpropagation in detail

Video 9

Programmer le réseau à 2 couches avec NumPy

Coding the 2-layer network in NumPy

Playlist

The whole Deep Learning series

10 videos, from the perceptron to the network with N layers

Videos in French. Click a line in your copy of the course to open the video.

Summary

1. A hidden layer draws several lines and creates new features; the output neuron combines them (XOR, moons).
2. Layer l: $Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$, $A^{[l]} = \sigma(Z^{[l]})$, with $W^{[l]}$ of shape $(n^{[l]}, n^{[l-1]})$. Without the non-linearity, the network is linear.
3. Backpropagation: $dZ^{[2]} = A^{[2]} - Y$, then $dZ^{[1]} = W^{[2]\top} dZ^{[2]} \odot A^{[1]}(1 - A^{[1]})$; $dW^{[l]} = \frac{1}{m} dZ^{[l]} A^{[l-1]\top}$.
4. `loss.backward()` computes exactly these gradients; `nn.Sequential` builds the network.

> **To go further**
>
> Coding a full neural network from scratch in NumPy (forward and backward, layer by layer) is a great exercise: I do it step by step on my YouTube channel Machine Learnia, in video 9 of the Deep Learning series (previous slide).

Next session: we master the whole PyTorch pipeline, line by line.